I Benchmarked 3 Compute Backends. The Results Were Interesting.

CPU vs Intel iGPU vs NVIDIA dGPU for embedding inference in **corvia**'s real-world RAG pipeline — and why the backend mattered less than I expected.

ONNX Runtime · OpenVINO · CUDA · Rust · nomic-embed-text-v1.5

github.com/chunzhe10/corvia

THE SETUP

One Laptop.
Three Backends.

corvia is organizational memory for AI agents. Its inference server embeds text into vectors for semantic search. I wanted to know which hardware backend to default to.

CPU
i7-12700H
14 cores / 20 threads
DDR5 39 GB

Intel iGPU
Iris Xe (96 EU)
OpenVINO 2025.3
Shared memory

NVIDIA dGPU
RTX 3060 Laptop
CUDA 12.8 / cuDNN 9.2
6 GB VRAM

MODEL UNDER TEST

nomic-embed-text-v1.5

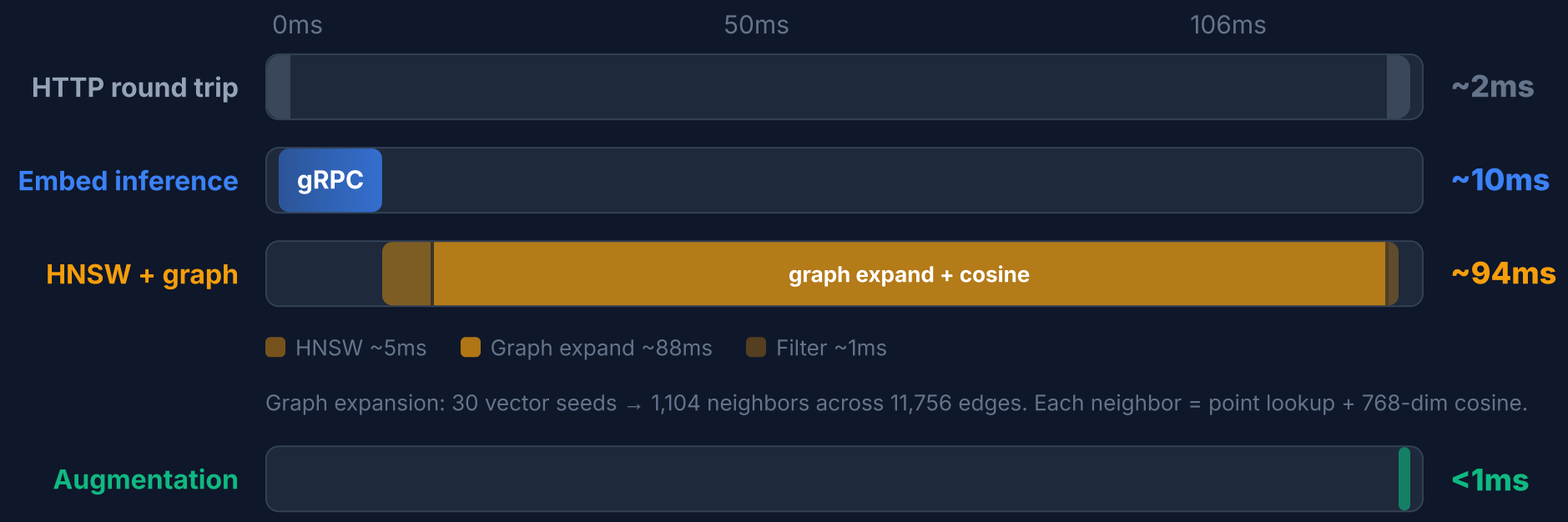
~137M parameters • 768 dimensions • ONNX format • via fastembed-rs

All three backends use the same ONNX Runtime — only the execution provider differs. Same model, same code path, different hardware.

WHERE THE TIME GOES

I Instrumented Every Stage.

Most benchmarks show total latency. I added per-stage timing to the server to see where time is actually spent. Here's a typical request on CUDA.



9%
of time is
embedding inference

89%
of time is
search + graph

2%
of time is
HTTP overhead

The embedding backend everyone benchmarks? It's less than 10% of the real-world latency. In a RAG pipeline, the retrieval layer — not the model — is what your users actually wait for.

THE NUMBERS

Per-Embed Latency Across All Inputs

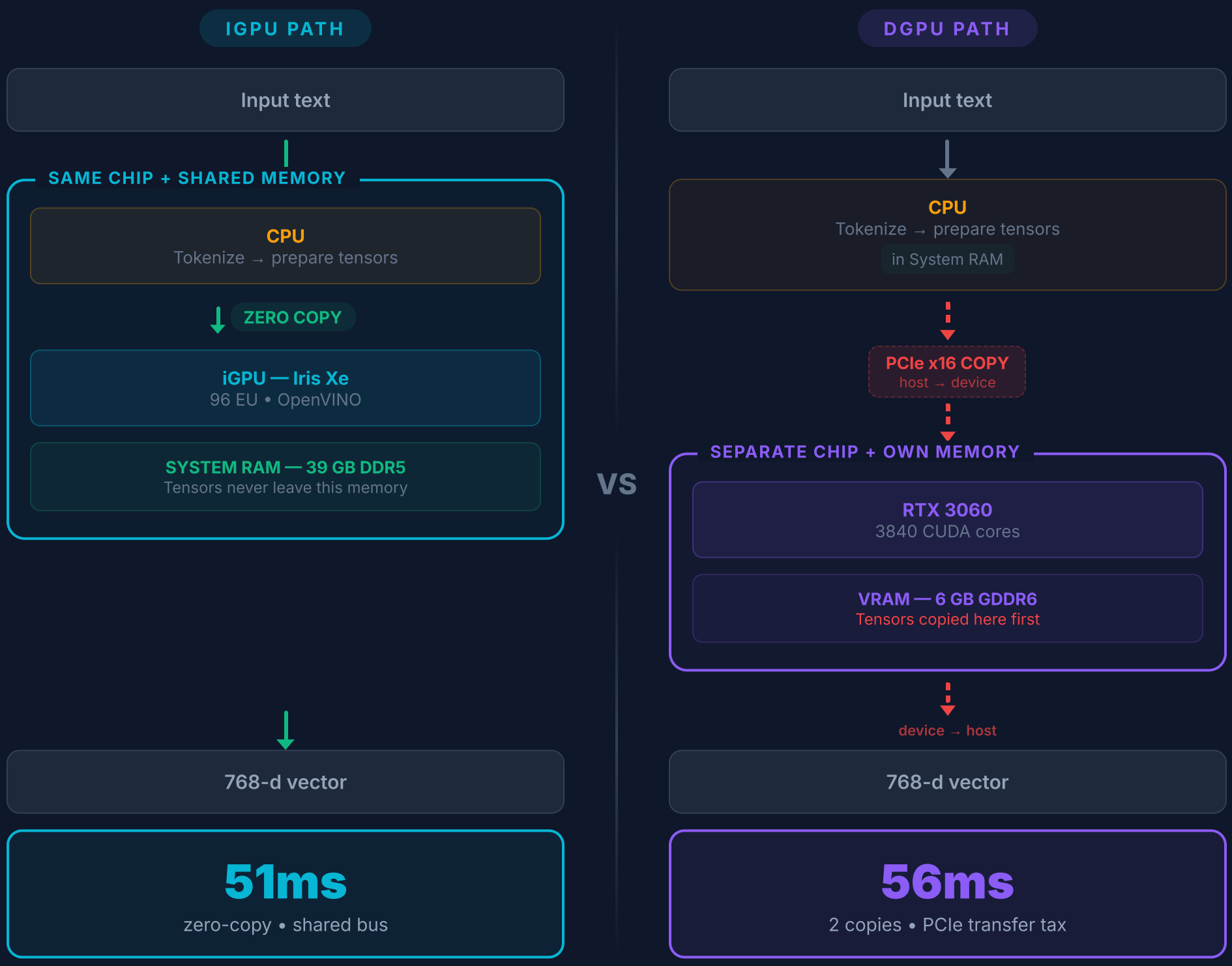
Lower is better. Each bar = mean of 5 runs. Error bars omitted for clarity (all ≤ ±11ms).

For medium and long texts, all three are within ~6ms of each other. The real gap only shows on short-text batches — and that's a CPU startup cost, not a sustained difference.

THE SURPRISE

The Integrated GPU Beat the Discrete GPU.

51ms vs 56ms. Not huge — but consistent. The diagram below shows why.



This does NOT mean iGPU > dGPU in general. For larger models, the RTX 3060's raw throughput wins. And in a 106ms pipeline, this 5ms gap is <5% of what users experience — optimizing retrieval matters more.

THE NUANCE

What the Numbers Don't Tell You

A benchmark is only as honest as its caveats. Here are the things I'd want to know before drawing conclusions from these numbers.

CPU short-text anomaly is misleading

527ms looks terrible, but it's ORT's batch startup cost spread over only 5 items. With 10+ items, CPU drops to 56ms/embed — nearly matching GPU backends.

Pipeline overhead dominates

~45-50ms of every request is CLI + HTTP + gRPC + HNSW search. Pure inference time is only ~5-10ms on GPU. The backend matters less than you'd think.

NVIDIA wins on different workloads

Bigger models, longer sequences, larger batch sizes — that's where 6 GB VRAM and 3840 CUDA cores pull ahead. This test used a 137M param model with small batches.

This is one machine

A laptop i7 + RTX 3060. Your hardware will differ. The pattern (shared memory wins for small models) likely generalizes, but the specific numbers won't.

The takeaway: these numbers are directionally useful, but your mileage will vary. Always benchmark on your own hardware.

WHAT IT TOOK

Behind
corvia use <backend>

One command to switch backends. Getting there was a different story.

0
PACKAGES

CPU
ORT default. Just works.

13
PACKAGES

Intel iGPU (OpenVINO)
5 Intel compute-runtime packages • 3 libtbb .debs cross-pulled from Ubuntu Noble (not in Debian Trixie) • 3 OpenVINO libs from Intel apt repo • manual symlinks because APT installs to /usr/lib/ but ORT checks /usr/lib/x86_64-linux-gnu/ • /dev/dri by-path symlinks for GPU discovery

5+
PACKAGES

NVIDIA dGPU (CUDA)
NVIDIA's apt repo uses SHA1-signed GPG keys — Debian Trixie rejects them. Had to install cublas, curand, cufft, cudart, cuDNN via pip instead • pip buries .so files deep in site-packages — manual symlink tree into /usr/lib/ • CDI spec auto-regen on driver version change

```
# After all that, the user just runs:
$ corvia use openvino  Reload complete.
$ corvia use cuda      Reload complete.
$ corvia use cpu       Reload complete.
```

The whole point: absorb the complexity so switching backends is one command, no restart, no config files.

KEY TAKEAWAYS

What I Learned
From These Numbers

📈

In RAG, retrieval matters more than the model.
89% of pipeline time was search + graph — not inference. Optimize the retrieval layer first.

⚡

Shared memory beats raw compute for small models.
If data transfer costs more than the computation, zero-copy wins.

🛠

Don't discard your iGPU.
It's free compute sitting idle. For embeddings, it can match or beat a discrete GPU.

🔌

Make backend switching painless.
Different hardware, different optimal backends. Let users switch at runtime.

Running embedding inference locally?
What backends have worked best for you?

Full benchmark report + reproduction steps on GitHub 👇